浙江大学

本科实验报告

课程名称：	实验 19 音乐播放器
姓　　名：	
学　　院：	信息与电子工程学院
系：	
专　　业：	信息工程
学　　号：	
指导教师：	屈民军，唐奕

2025 年 5 月 16 日

专业：信息工程
姓名：
学号：
日期：2025.5.16
地点：东四

浙江大学实验报告

课程名称：数字系统设计实验
指导老师：屈民军，唐奕
成绩：
实验名称：实验 19 音乐播放器

一、实验目的

1．掌握音符产生的方法，了解 DDS 技术的应用

2．了解音频解码的应用

3．掌握系统“自顶而下”的数字系统设计方法

二、概述与实验任务

1．概述：音符产生的原理

（1）音符最主要的要素是“音的高低”和“音的长度”，高低指每个音符的频率不同，长短用拍子表示，一拍是一个相对时间度量单位

（2）实验中，用{note,duration}表示音符，note、duration 为 6 位二进制数，duration 单位为 1/48s，实验用 128*128bits 的 song_rom 存放

（3）FPGA，正弦信号以一定的速率送给音频编解码系统，由音频编解码系统将样品转换为电压，驱动扬声器发声。正弦信号的产生用 DDS 技术

音频编解码期取样频率为 48kHz，用 lab15 的 Sine ROM，已知音符 f 就可以求出相位增量 k，将 fo=48kHz，m=12 代入（6.15）满足

$$fo = (fc * K)/2^m \text{.....（6.15）}$$

$$k = f(Hz)/11.7 \text{.....(6.29)}$$

为了精确得到音符频率，用 20 位二进制表示 k，高 10 位为二进制整数

（4）由于音符个数有限，采用查找表方法实现（6.29）的出发比较方便，一

次你实验中还需要 Frequency ROM 存放 k，大小为 64*24bits

2．实验任务

设计一个音乐播放器

（1）可以播放四首乐曲，设置 play/pause_button、next_button、reset 三个按键，按 play/pause_button，音乐在播放和暂停之间切换，按 next_button 播放下一首

（2）LED0 指示播放情况，LED2 和 LED3 指示当前乐曲序号

三、实验设计原理与 Verilog 代码

3.1 顶层代码设计

1．根据实验任务可将系统划分为时钟管理模块（DCM）、按键处理、主控制器、乐曲读取、音符播放、同步✿化电路、节拍基准产生器和音频编解码接口电路等子模块

（1）时钟管理模块（DCM）产生 100MHz 的系统时钟 sys_clk 和 12.5MHz 的音频时钟 audio_clk

（2）主控制器（mcu）模块接收案件信息，通知 song_reader 是否播放和曲目

（3）乐曲读取（song_reader）根据 mcu 的要求，逐个取出音符信息{note,duration}送给 note_player 播放，完毕后回复 mcu 结束信号 song_done

（4）音符播放接收所需的音符，duration 内以 48kHz 速率送出样品给音频编解码接口模块，结束后向 song_reader 发送 note_done 索取新的音符

（5）音频编解码接口模块负责将音符的正弦波样品转换为串行输出并发送给音频编解码芯片 ADAU1761，进行 AD 转换并放大，最后播放

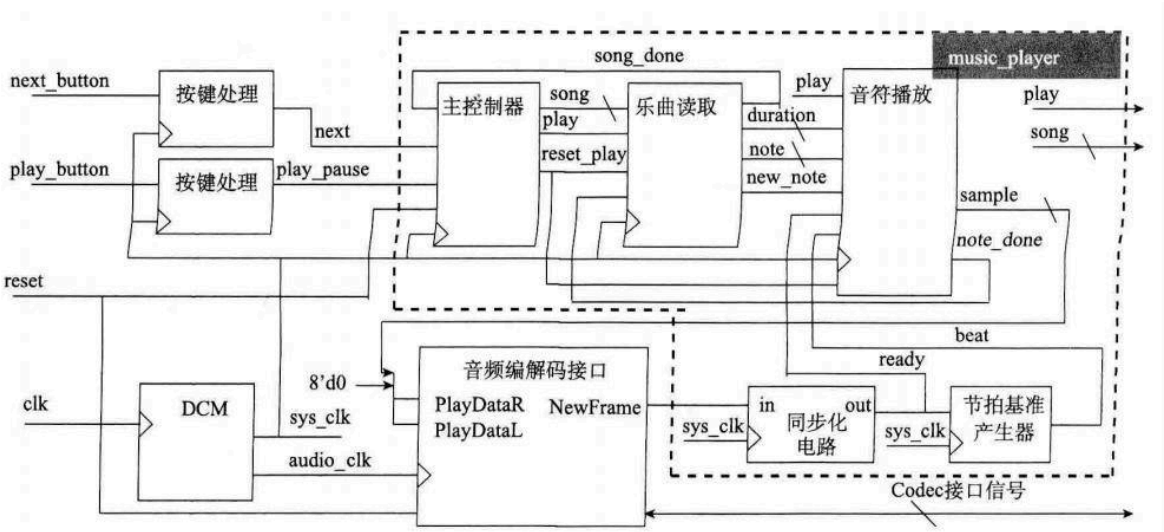
（6）同步化电路：音频编解码模块和系统时钟不同，协调两部分电路

（7）节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号（beat），而 ready

信号频率为 48kHz，节拍基准产生器为 1000 分频的分频器

（8）按键处理模块完成输入同步化、防颤动和脉冲变换等功能

2．顶层框图



3.2 主控制器 mcu 设计

1．任务

响应按键信息和控制系统播放两大任务

2．端口含义

表 6.11　主控制模块 mcu 的端口含义

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play_pause	Input	来自按键处理模块的“播放/暂停”控制信号，一个时钟周期宽度的脉冲
next	Input	来自按键处理模块的“下一曲” 控制信号，一个时钟周期宽度的脉冲
play	Output	输出控制信号，高电平表示播放，控制 song_reader 模块是否要播放
reset_play	Output	时钟周期宽度的高电平复位脉冲 reset_play，用于同时复位模块 song_reader 和 note_player
song_done	Input	song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示一曲播放结束
song[1:0]	Output	当前播放乐曲的序号

3．mcu 结构框图

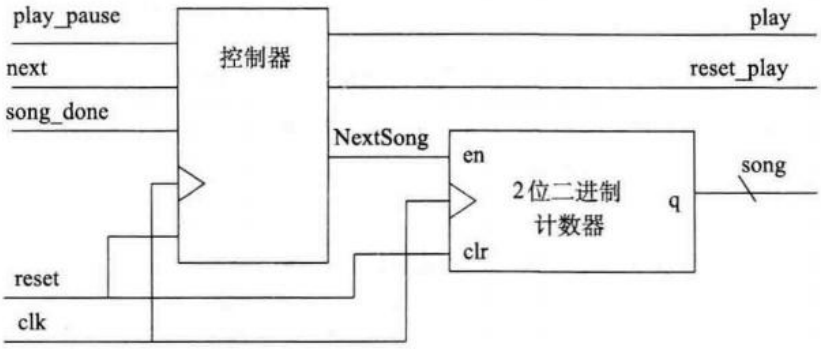


图 6.45　mcu 的结构框图

（1）控制器工作流程：由 RESET、PLAY、PAUSE、NEXT 四种状态。系统复位后经过 RESET 后进入 PAUSE，play_pause 使系统在 PLAY 和 PAUSE 之间转换，在 PLAY 或者 PAUSE 状态下，按下 next_button 后系统进入 NEXT 状态，同时输出 reset_play 复位后续的 song_reader 和 note_player 模块，同时 Next Song 使得乐曲序号加 1，进入下一曲播放；或者在 PLAY 状态下播放结束，song_done==1 则进入 RESET，同时输出 reset_play 复位后续的 song_reader 和 note_player 模块。

（2）流程图如下

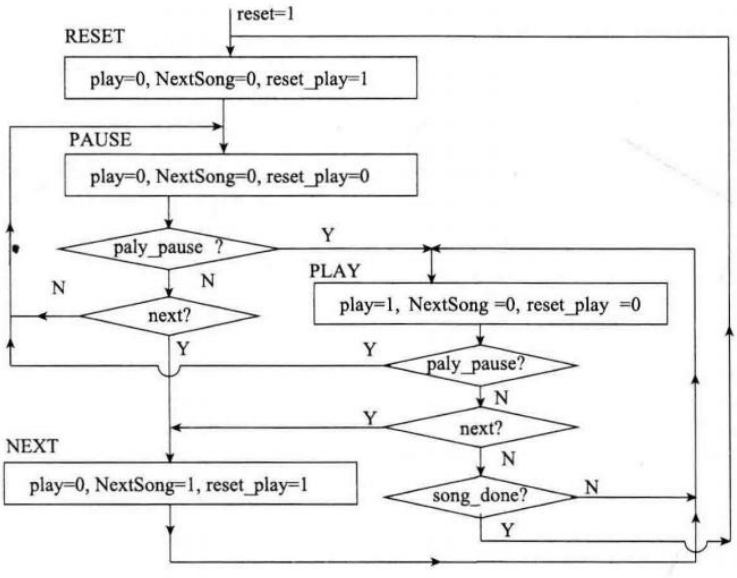


图 6.46　mcu 控制器的算法流程图

4．mcu 代码

```
module mcu(play_pause,next,song_done,reset,clk,play,reset_play,song);
    input clk,reset,play_pause,next,song_done;
    output play,reset_play;
    output [1:0] song;
    //控制器部分
    parameter RESET=0,PAUSE=1,PLAY=2,NEXT=3;//编码
    reg [1:0] state,next_state;
    reg play,reset_play,NextSong;
    //控制器异步置零
    always @(posedge clk)
        begin
            if(reset) state=RESET;
            else state=next_state;
        end

    //控制器部分,根据现态和输入变量决定次态
    always@(*)
        begin
            play=0;
            NextSong=0;
            reset_play=1;
        case(state)
            RESET:
                begin
                    play=0;
                    NextSong=0;
                    reset_play=1;
                    next_state=PAUSE;
                end
            PAUSE:
                begin
                    play=0;
                    NextSong=0;
                    reset_play=0;
                    next_state=(play_pause==1)?PLAY:((next==1)?NEXT:PAUSE);
                end
            PLAY:
                begin
                    play=1;
                    NextSong=0;
                    reset_play=0;
                    next_state=(play_pause==1)?PAUSE:((next==1)?NEXT:((song_done==1)?RESET:PLAY));
                end
            NEXT:
                begin
                    play=0;
                    NextSong=1;
                    reset_play=1;
                    next_state=PLAY;
                end
        endcase
        end
    //2位2进制计数器
    counter_n #(.n(4),.counter_bits(2)) counter1(.clk(clk),.r(reset),.en(NextSong),.co(),.q(song));
endmodule
```

3.3 乐曲读取模块 song_reader 设计

1．任务

（1）根据 mcu 要求，选择播放乐曲

（2）响应 note_player 模块要求，从 song_rom 中逐个取出音符{note,duration}发送给 note_player 模块

（3）判断乐曲是否播放完毕，若播放完毕，则回复 mcu

2．端口含义

表 6.12　乐曲读取模块 song_reader 的端口含义

引脚名称	I/O	引脚说明
clk	Input	100MHz 时钟信号
reset	Input	复位信号，高电平有效
play	Input	来自 mcu 的控制信号，高电平要求播放
song[1:0]	Input	来自 mcu 的控制信号，当前播放乐曲的序号
note_done	Input	即模块 note_player 的应答信号，一个时钟周期宽度的脉冲，表示一个音符播放结束并索取新音符
song_done	Output	给 mcu 的应答信号，当乐曲播放结束，输出一个时钟周期宽度的脉冲，表示乐曲播放结束
note[5:0]	Output	音符标记
duration[5:0]	Output	音符的持续时间
new_note	Output	给模块 note_playe 的控制信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放

3．song_rom 模块介绍

（1）是一个只读存储器，用来存放乐曲，容量为 2^7*12bits，共存放四首乐曲，每首乐曲占用 2^5*12bits，每首乐曲最长由 32 个音符组成。

（2）song_rom 高 2 位地址决定哪首乐曲，低 5 位地址决定哪个音符，song_rom 输出的音符信息为 12 位二进制，高 6 位为 note，低 6 位为 duration

4．song_reader 结构框图

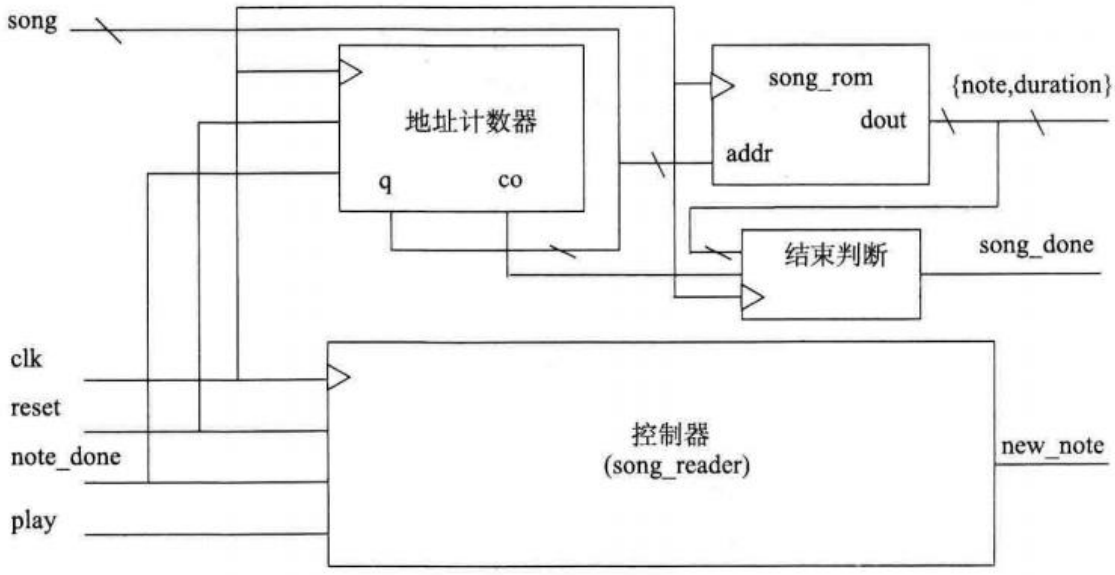
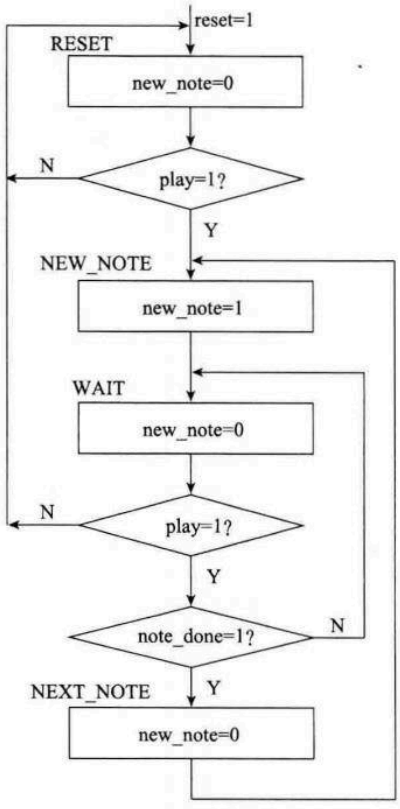


图 6.47　song_reader 的结构框图

（1）控制器：复位后一直处于 RESET 等待 mcu 发出 play 命令，接收到 play 后进入 NEW_NOTE 状态，输出 new_note=1 要求 note_player 播放，之后进入 WAIT 状态，当 note_player 播放完毕后，发出 note_done 索取下一个音符，note_done 一方面让地址计数器递增，另一方面让状态转移到 NEXT_NOTE 状态，输出 new_note=0 通知 note_player 有新音符。

①NEXT_NOTE 状态存在还是为了**延迟一个时钟周期**输出信号，以配合 song_rom 的读取要求

②状态图



（2）地址计数器：5 位二进制计数器，note_done 为使能端，状态 q 为 song_rom 的低 5 位地址，song[1:0]为 song_rom 的高 2 位地址

（3）当地址计数器进位或者 duration=0 时，表示乐曲结束，应该输出一个 clk 周期的高电平脉冲信号 song_done。由于 duration=0 持续时间可能不止一个时

钟周期，因此结束判断模块需使用脉冲变换电路实现

5．代码

```
module song_reader(song,clk,reset,note_done,play,note,duration,song_done,new_note);
    input [1:0] song;
    input clk,reset,note_done,play;
    output [5:0] note,duration;
    output song_done,new_note;

    reg [5:0] note,duration;
    //初始化赋值（观察到一开始是红线）
    initial begin
        note=0;
        duration=0;
    end
    //控制器部分
    parameter RESET=0,NEW_NOTE=1,WAIT=2,NEXT_NOTE=3;//编码
    reg [1:0] state,next_state;
    reg new_note;
    //控制器异步置零
    always @(posedge clk)
        begin
            if(reset) state=RESET;
            else state=next_state;
        end

    //控制器部分,根据现态和输入变量决定次态
    always@(*)
        begin
            new_note=0;
        case(state)
            RESET:
                begin
                    new_note=0;
                    next_state=(play==1)?NEW_NOTE:RESET;
                end
            NEW_NOTE:
                begin
                    new_note=1;
                    next_state=WAIT;
                end
            WAIT:
                begin
                    new_note=0;
                    next_state=(play==1)?((note_done==1)?NEXT_NOTE:WAIT):RESET;
                end
            NEXT_NOTE:
                begin
                    new_note=0;
                    next_state=NEW_NOTE;
                end
        endcase
        end
    //地址计数器
    wire co;
    wire [6:0] addr;
    wire [11:0] dout;
    //addr低5位 计数器状态q
    counter_n #(.n(32),.counter_bits(5)) counter2(.clk(clk),.r(reset),.en(note_done),.co(co),.q(addr[4:0]));
    counter_n #(.n(32),.counter_bits(5)) counter2(.clk(clk),.r(reset),.en(note_done),.co(co),.q(addr[4:0]));
    //高两位song地址
    assign addr[6:5]=song[1:0];
    //不太理解为什么dout后面不能直接跟{note,duration} 会报错！
    song_rom song_rom1(.clk(clk),.dout(dout),.addr(addr));
    always@(*) {note,duration}=dout;
    //结束判断
    pulse_variation pulse_variation1(.in(co==1||duration==0),.clk(clk),.out(song_done));
endmodule
```

3.4 音乐播放模块 note_player（核心模块）

1．任务

（1）从 song_reader 模块接收需要播放的音符{note,duration}

（2）根据 note 找出 DDS 的相位增量 K

（3）以 48kHz 速率从 SineROM 中取出正弦样品送给音频编解码器接口

（4）当一个音符播放完毕，向 song_reader 索取新的音符

2．端口含义

表 6.13　note_player 模块的端口含义

引脚名称	I/O	引脚说明
clk	Input	系统时钟信号，外接 sys_clk
reset	Input	复位信号，高电平有效，外接 mcu 模块的 reset_play
play_enable	Input	来自 mcu 模块的 play 信号，高电平表示播放
note_to_load[5:0]	Input	来自 song_reader 模块的音符标记 note，表示需播放的音符
duration_to_load[5:0]	Input	来自 song_reader 模块的音符持续时间 duration，表示需播放音符的音长
load_new_note	Input	来自 song_reader 模块的 new_note 信号，一个时钟周期宽度的高电平脉冲，表示新的音符需播放
note _ done	Output	给 song_reader 模块的应答信号，一个时钟周期宽度的高电平脉冲，表示音符播放完毕
sampling_pulse	Input	来自同步化电路模块的 ready 信号，频率 48kHz，一个时钟周期宽度的高电平脉冲，表示索取新的正弦样品
beat	Input	定时基准信号，频率为 48Hz 脉冲，一个时钟周期宽度的高电平脉冲
sample [15:0]	Output	正弦样品输出

3．结构框图

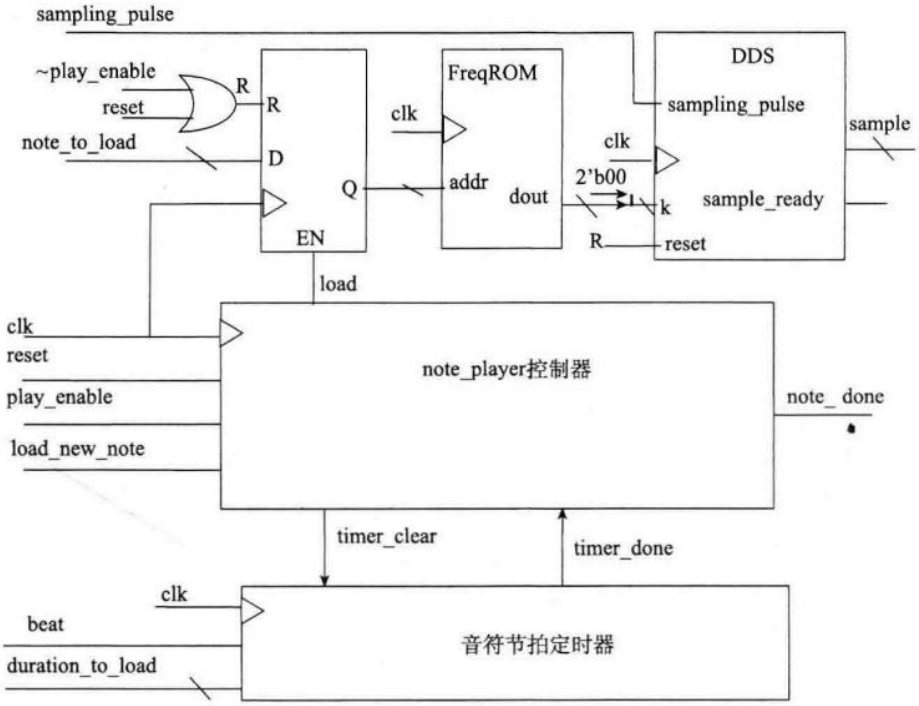


图 6.50　note_player 模块的结构框图

（1）控制器：与 song_reader 相连接，读取音符信息并据此从 Frequency ROM 中读取 k 送给 DDS 模块，并控制音符播放时间。在复位或未播放时，处于 RESET 或 PLAY，此时高电平 reset 或低电平 play_enable 均使得 D 寄存器清 0，使 k=0，不会输出正弦样品。当 play_enable 高电平时，进入 PLAY 状态，当一个音符播放结束，进入 DONE 状态，note_done=1，向 song_reader 索要新音符，此时 song_reader 输出一个 new_note 使进入 LOAD 状态，读取音符并进入 PLAY 状态

（2）音符定时器：6 位二进制计数器，beat、timer_clear 为使能和清 0 信号，定时时间为 duration_to_load 个 beat 周期，timer_done 为定时结束标志

（3）DDS：lab15 中 K 为 22 位二进制数，因此需要将 FreqROM 输出的 20 位相位增量高位加 2 个 0 接入 DDS

4．代码

```
module note_player(clk,reset,play_enable,note_to_load,duration_to_load,load_new_note,note_done,sampling_pulse,beat,sample,sample_ready);
    input clk,reset,play_enable,load_new_note,sampling_pulse,beat;
    input [5:0] note_to_load,duration_to_load;
    output note_done,sample_ready;
    output [15:0] sample;

    //控制器
    reg timer_clear,load,note_done;
    reg [1:0] state,next_state;
    reg timer_done;

    parameter RESET=0, WAIT=1, DONE=2, LOAD=3;//编码
    always@(posedge clk) begin
        if(reset) state=RESET;
        else state=next_state;
    end
    always@(*) begin
        timer_clear=0;
        load=0;
        note_done=0;//默认为0
        case (state)
            RESET:
                begin
                    timer_clear=1;
                    load=0;
                    note_done=0;
                    next_state=WAIT;
                end
            WAIT:
                begin
                    timer_clear=0;
                    load=0;
                    note_done=0;
                    if(!play_enable) next_state=RESET;
                    else if(timer_done) next_state=DONE;
                    else if(load_new_note) next_state=LOAD;
                    else next_state=WAIT;
                end
            DONE:
                begin
                    timer_clear=1;
                    load=0;
                    note_done=1;
                    next_state=WAIT;
                end
            LOAD:
                begin
                    timer_clear=1;
                    load=1;
                    note_done=0;
                    next_state=WAIT;
                end
        endcase
    end
```

```
    //节拍定时器(实则为定时器 n=duration_to_load counter_bits=6
    reg [5:0] q=0;
    always @(posedge clk)
        begin
            timer_done=(q==(duration_to_load-1))&&beat;//定时结束
            if(timer_clear) q=0;
            else if(beat) q=q+1;
            else q=q;
        end

    //其他功能单元
    //注意r后面是按位或 不是条件或
    wire [5:0] temp_q;
    dffre #(.n(6))ff1(.d(note_to_load),.en(load),.r(~play_enable|reset),.clk(clk),.q(temp_q));
    wire [19:0] temp_dout;
    frequency_rom frequency_rom_inst(.clk(clk),.dout(temp_dout),.addr(temp_q));
    //注意端口的引用
    dds dds_inst(.k({2'b00,temp_dout}),.clk(clk),.reset(~play_enable|reset),.sampling_pulse(sampling_pulse),.sample(sample),.new_sample_ready(sample_ready));
endmodule
```

3.5 同步化电路

1．由于音频编解码接口模块和其他模块采用不同的时钟，因此需要进行同步化

2．音频编解码接口模块的输出信号 New Frame 的脉冲宽度为一个 audio_clk 时钟周期需同步化处理，产生与 sys_clk 同步且脉冲为一个 sys_clk 时钟周期的信号 ready

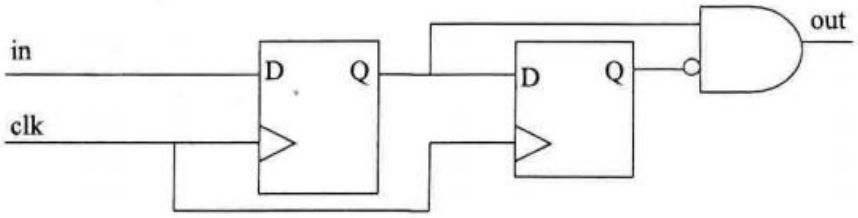


图 6.52　同步化电路

3.6 时钟管理模块（DCM）

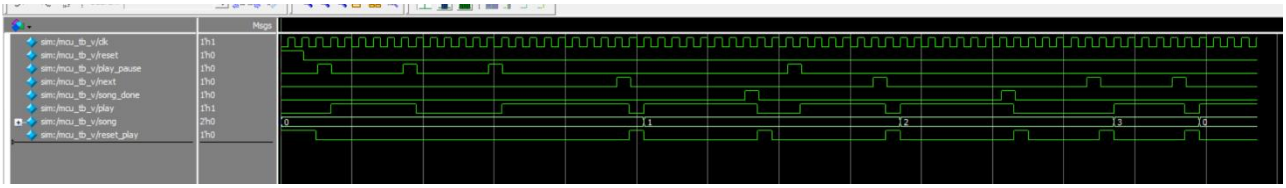
IP 内核时钟管理模块的输入时钟 clk 频率为 100MHz，产生 100MHz 的系统时钟和 12.5MHz 的音频时钟

3.7 本实验以 BlackBox 方式提供音频编解码接口模块与按键处理模块

四、ModelSim 仿真结果和分析

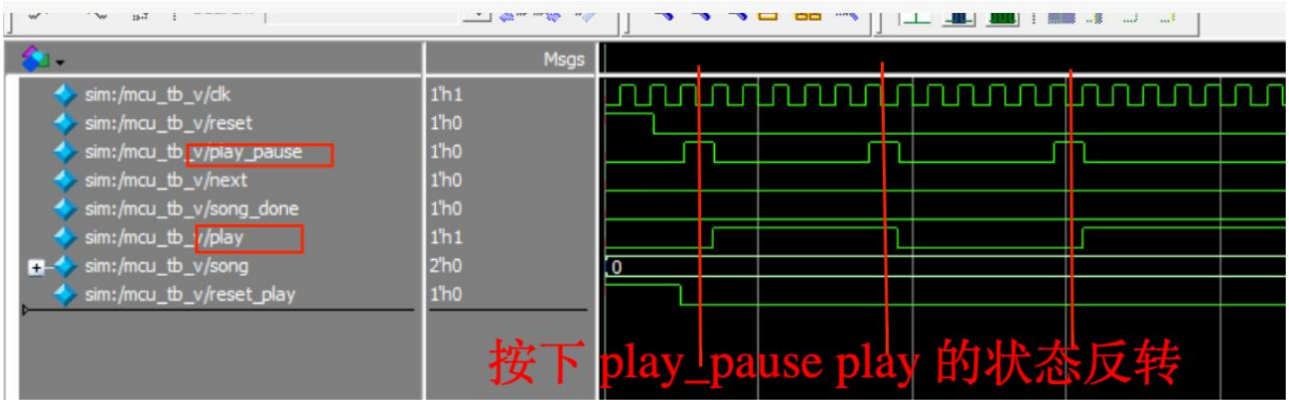
4.1 mcu 模块仿真

1．执行 mcu.tb 后得到的 wave

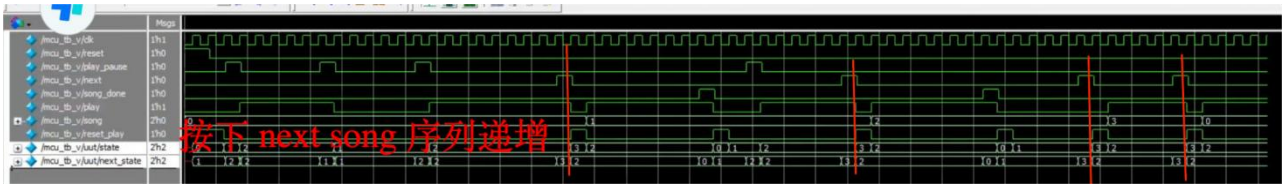


2．具体分析：

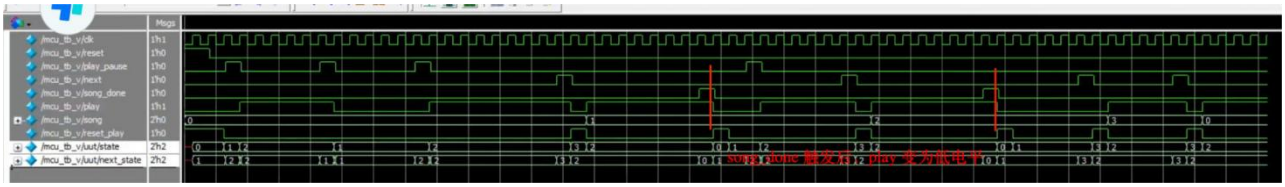
（1）INPUT：play_pause，每次按下，play 的状态都会改变一次（play 的电平状态反转）



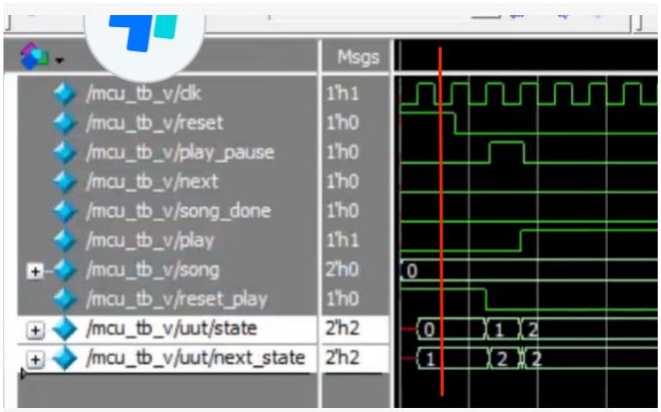
（2）INPUT：next，每次按下 next，song 的序列数会加 1，切换下一首歌曲



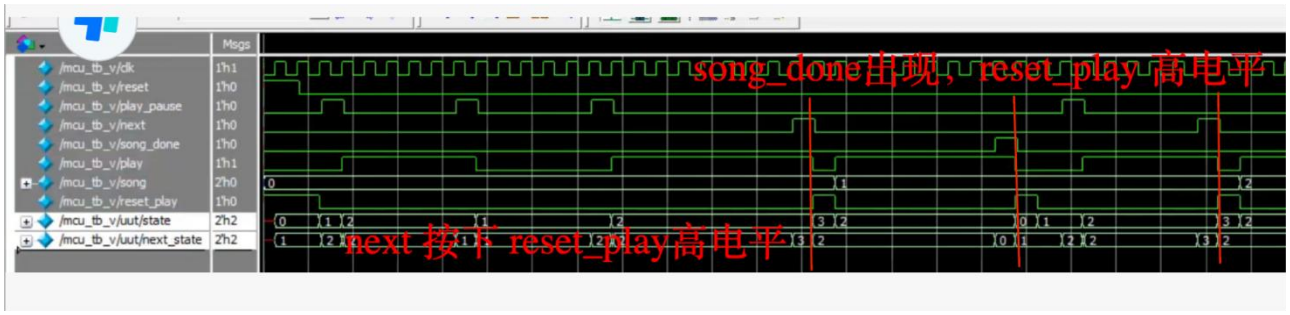
（3）INPUT：song_done 每次“按下”（实际上是 song_reader 的应答信号），play 处于低电平，停止播放



（4）INPUT：reset 每次按下，play 处于低电平，处于 0 状态（RESET 状态）



（5）OUTPUT：reset_play，每次歌曲要结束（无论是播放完毕，song_done=1，还是切换歌曲，next=1）时，reset_play 都会变为高电平



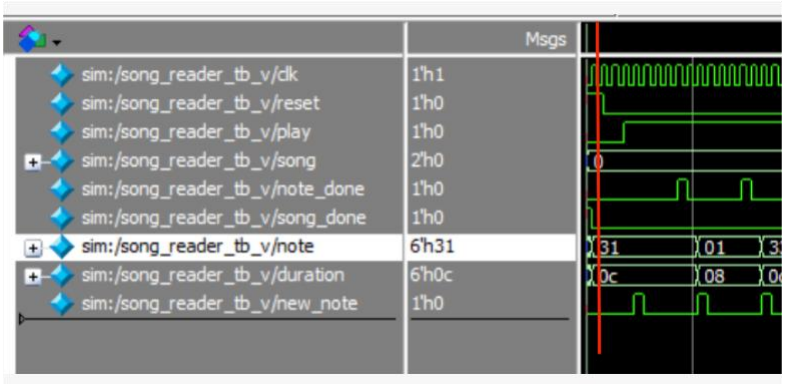
4.2 song_reader 模块仿真

1．执行 song_reader.tb 后得到的 wave（图二为转化为十进制）

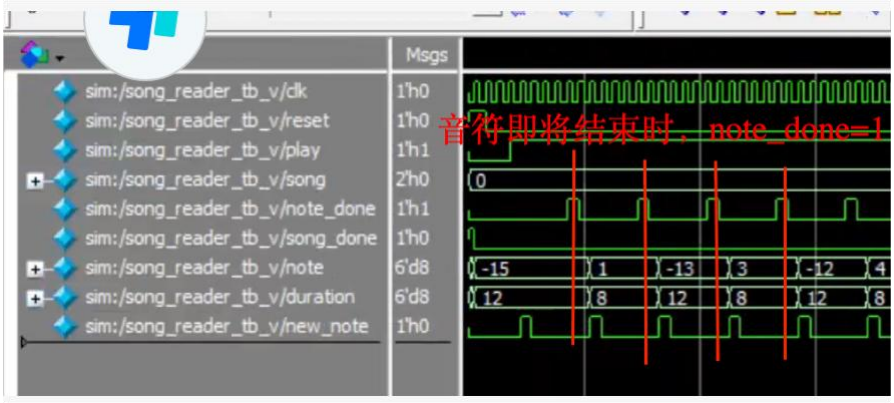


2．具体分析：

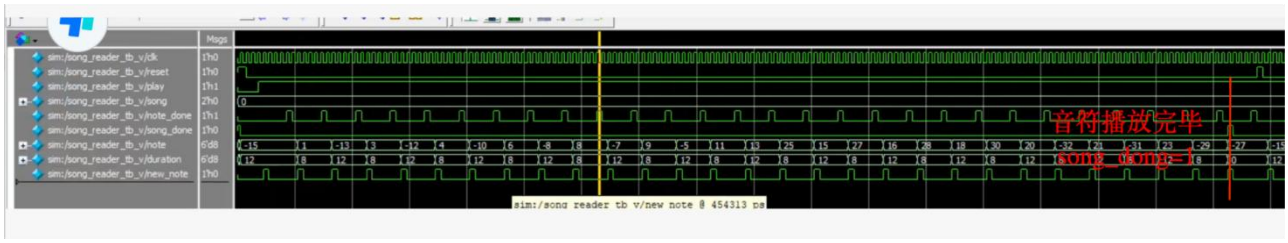
（1）INPUT：reset，每次按下 reset（高电平有效）后，play=0，不播放



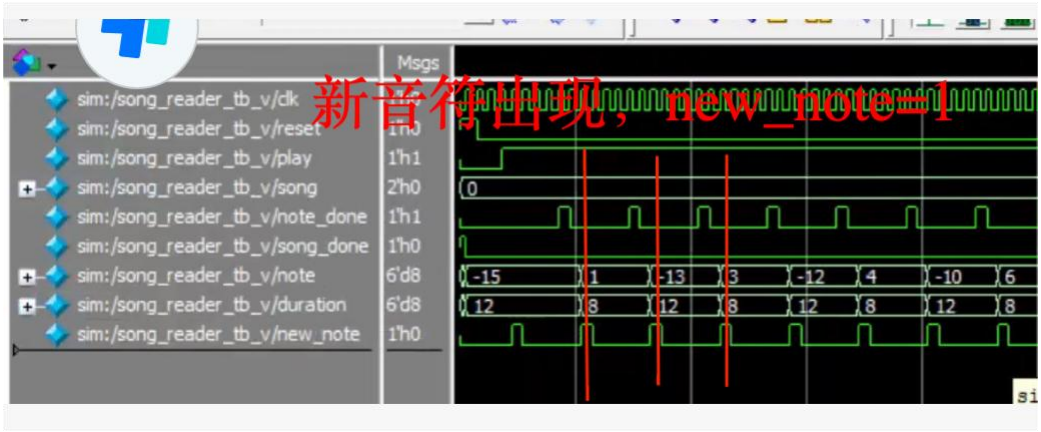
（2）INPUT：note_done，当音符播放结束后，note_done=1



（3）OUTPUT：song_done，一首歌曲所有音符播放结束后，变高电平

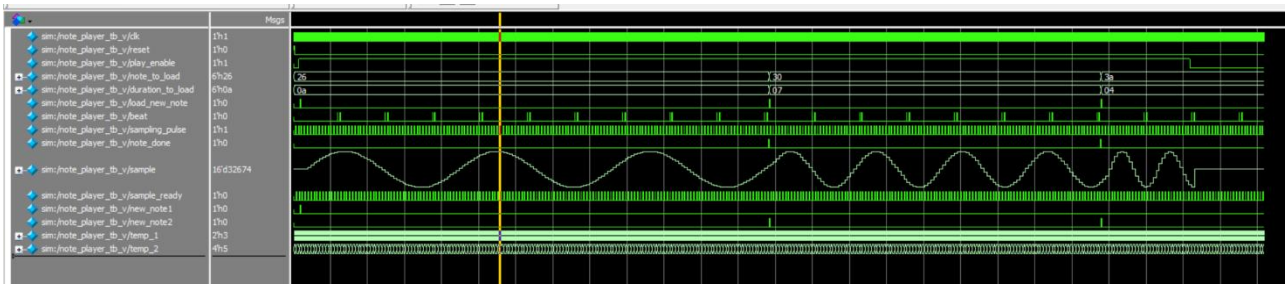


（4）OUTPUT：new_note，上一个音符播放完毕后，new_note=1 变为高电平，表示新的音符需要播放



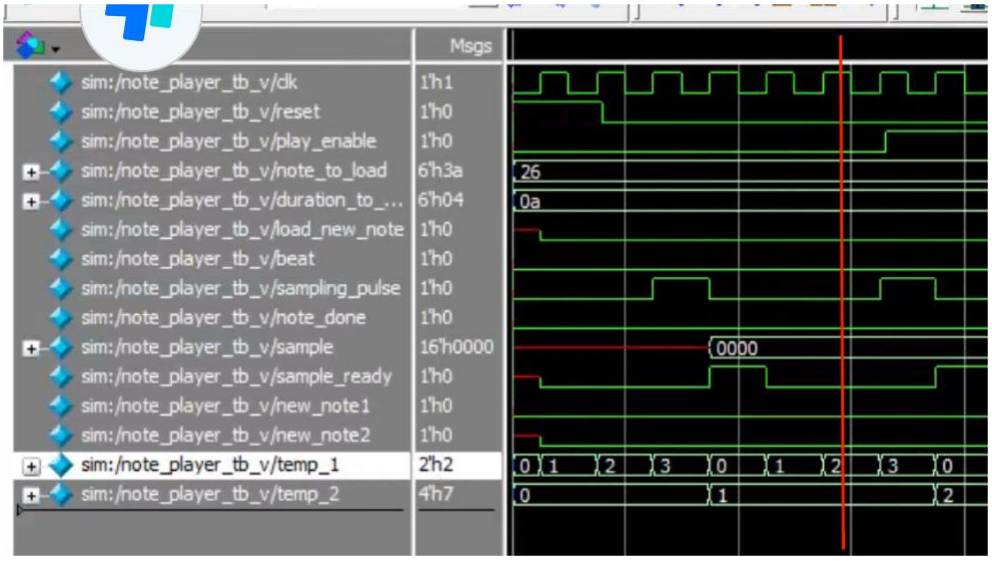
4.3 note_player 模块仿真

1．执行 note_player.tb 后得到的 wave

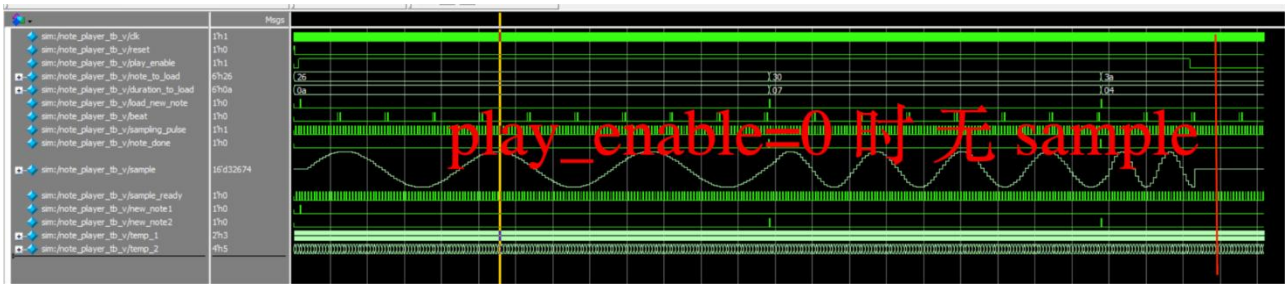


2．具体分析

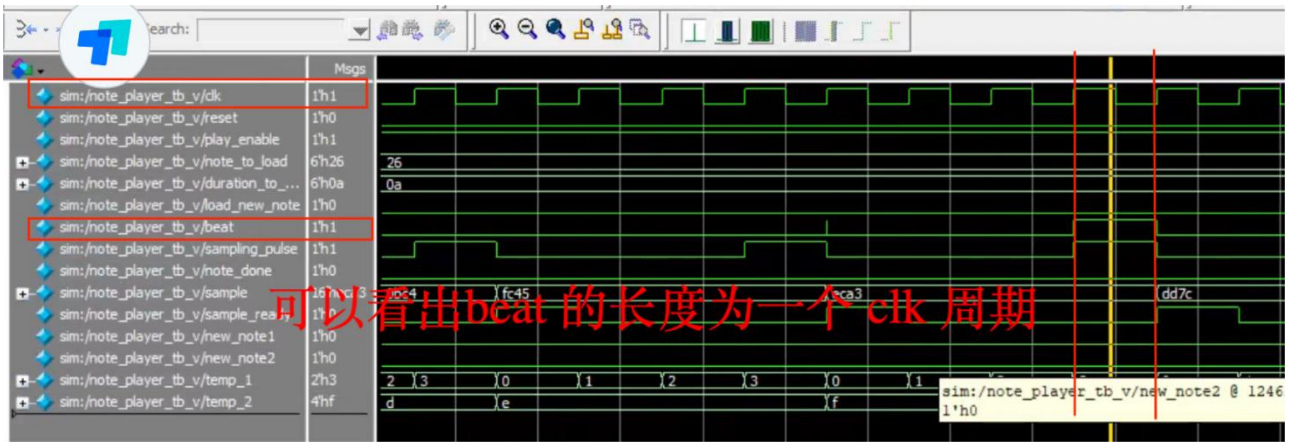
（1）INPUT：reset 按键，外接 mcu 的 reset_play，play_enable=0,sample=0,没有正弦信号采样和输出



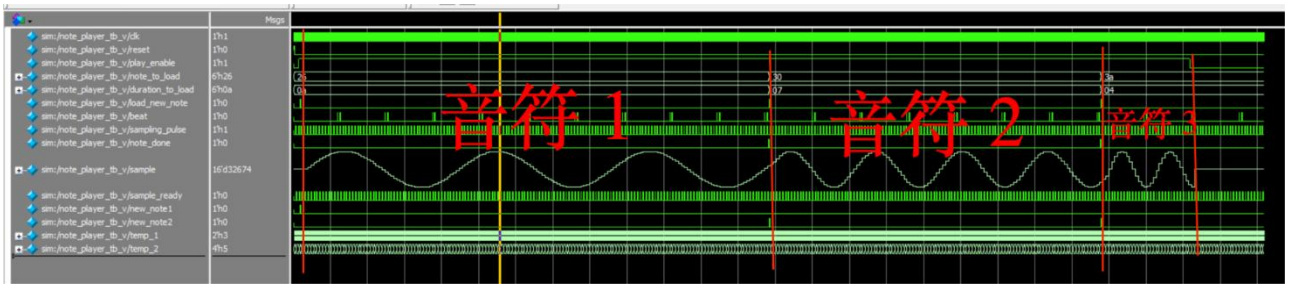
（2）INPUT：play_enable，play_enable=1 开始播放音符，为 0 时，sample=0，没有正弦信号输出



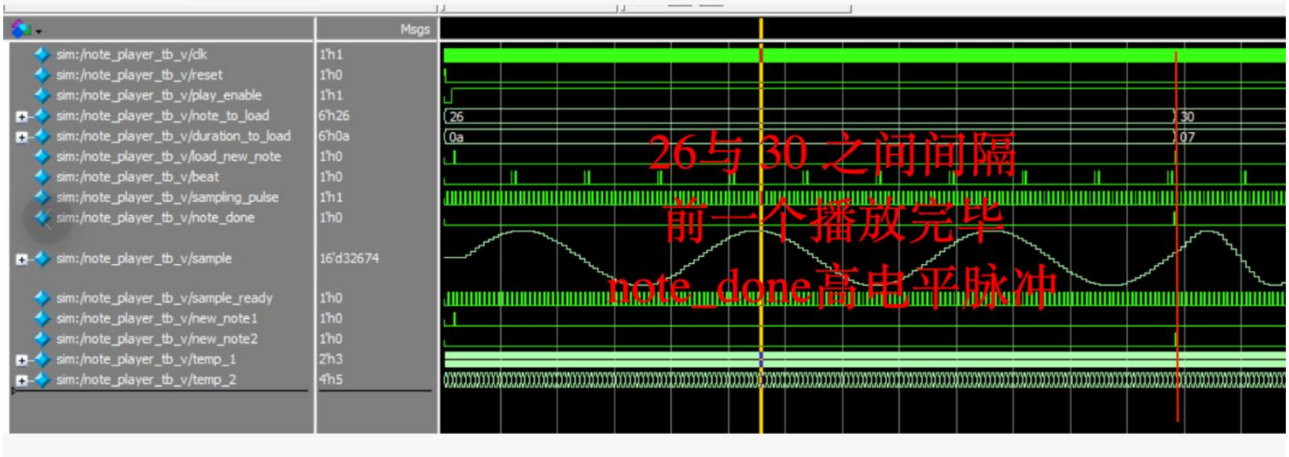
（3）INPUT：beat，定时基准信号，一个时钟周期宽度的高电平脉冲



（4）OUTPUT：sample[15:0] 正弦样品输出，不同的音符得到的正弦信号频率时不同的

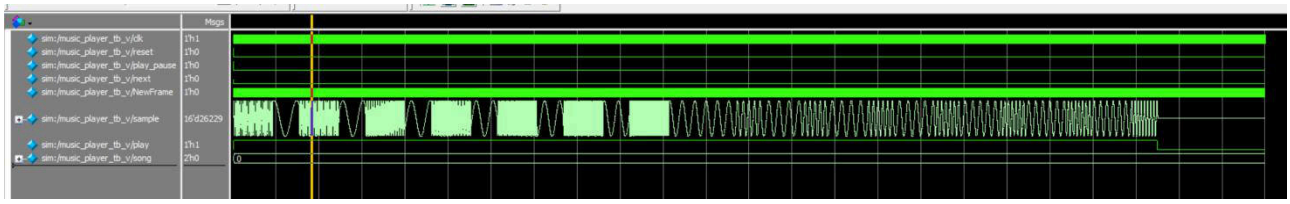


（5）OUTPUT：note_done，当音符播放完毕，输出一个高电平脉冲



4.4 music_player 模块仿真

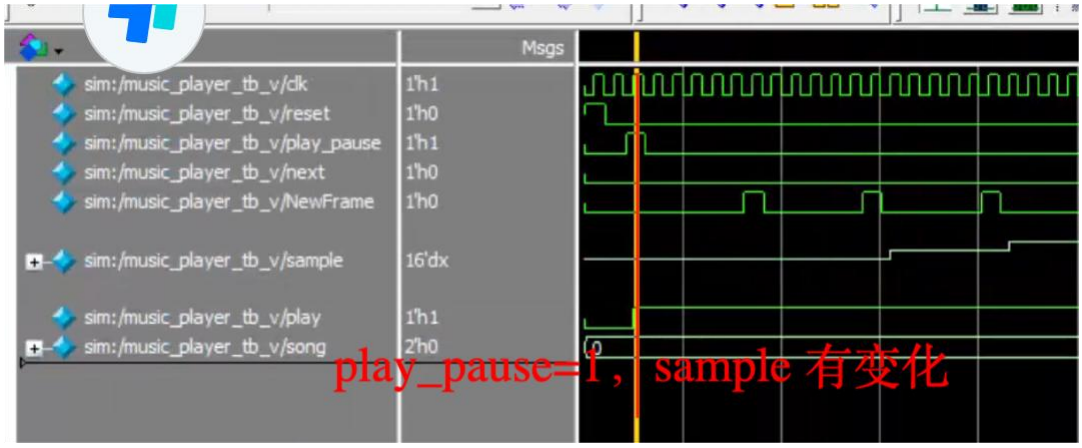
1．执行 music_player.tb 后得到的 wave



2．具体分析

（1）INPUT：reset 按键，复位输入，高电平有效

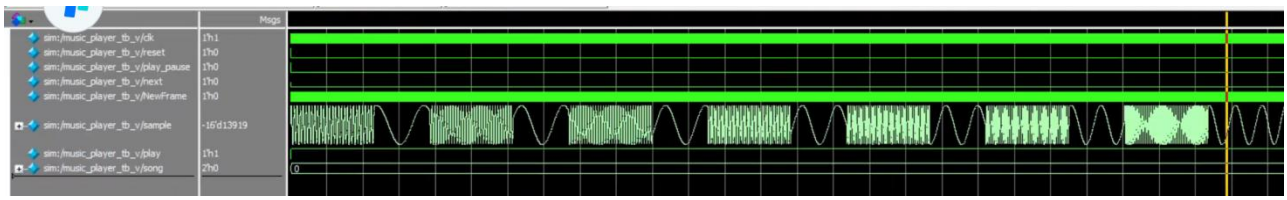
（2）INPUT：play_pause，按下 play_pause 后，歌曲开始播放，sample 有正弦输出



（3）INPUT：next，按下 next 后，切换歌曲，sample 的正弦波频率变化

（4）OUTPUT：song，指示歌曲的序号，波形中只播放了第 0 首歌曲

（5）OUTPUT：sample，不同音符的正弦波频率不同，一首歌曲中 sample 的正弦波频率在不断变换，因为音符在不断改变

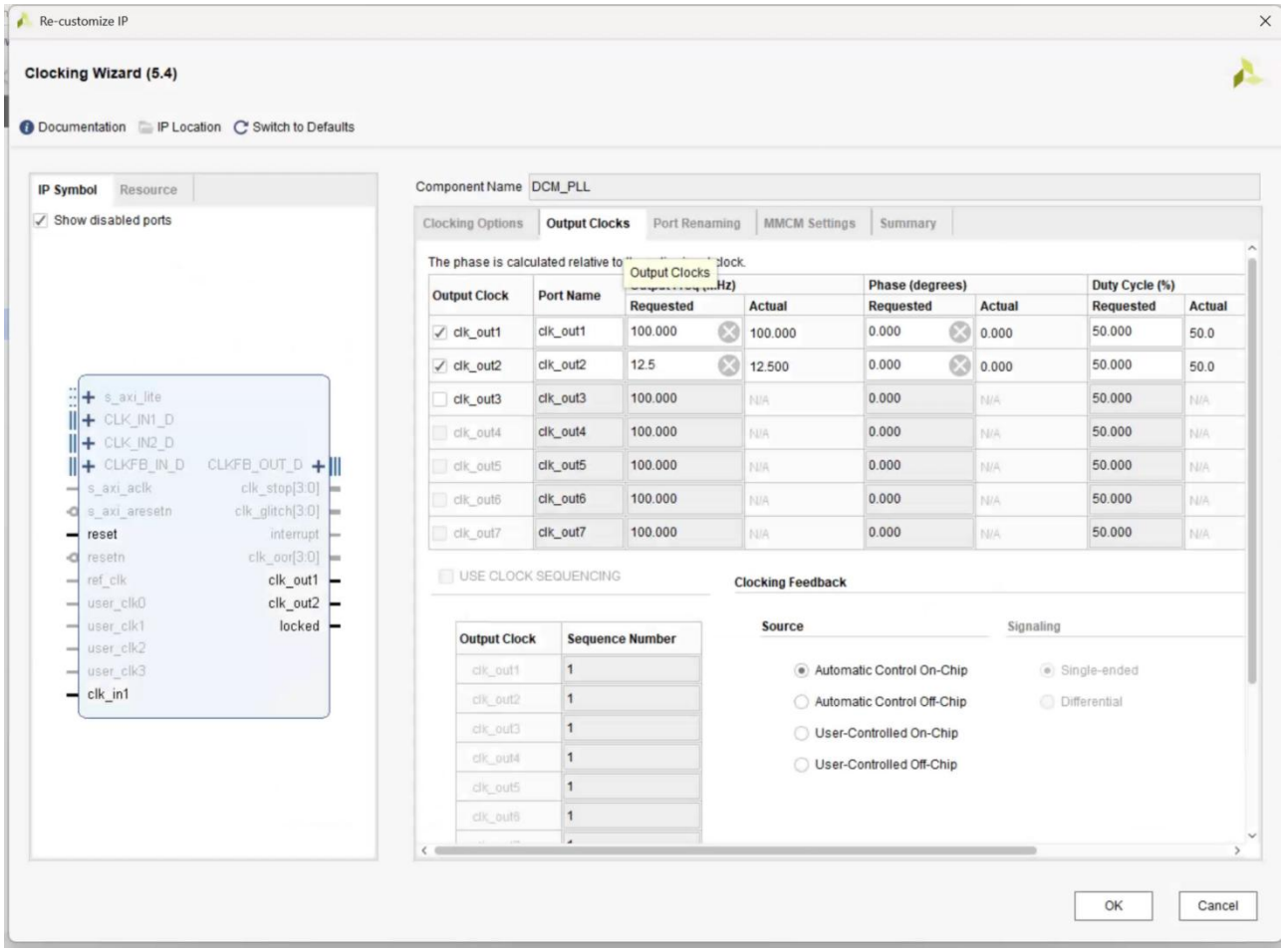


五、实验结果和开发板测试

1．vivado 工程文件建立

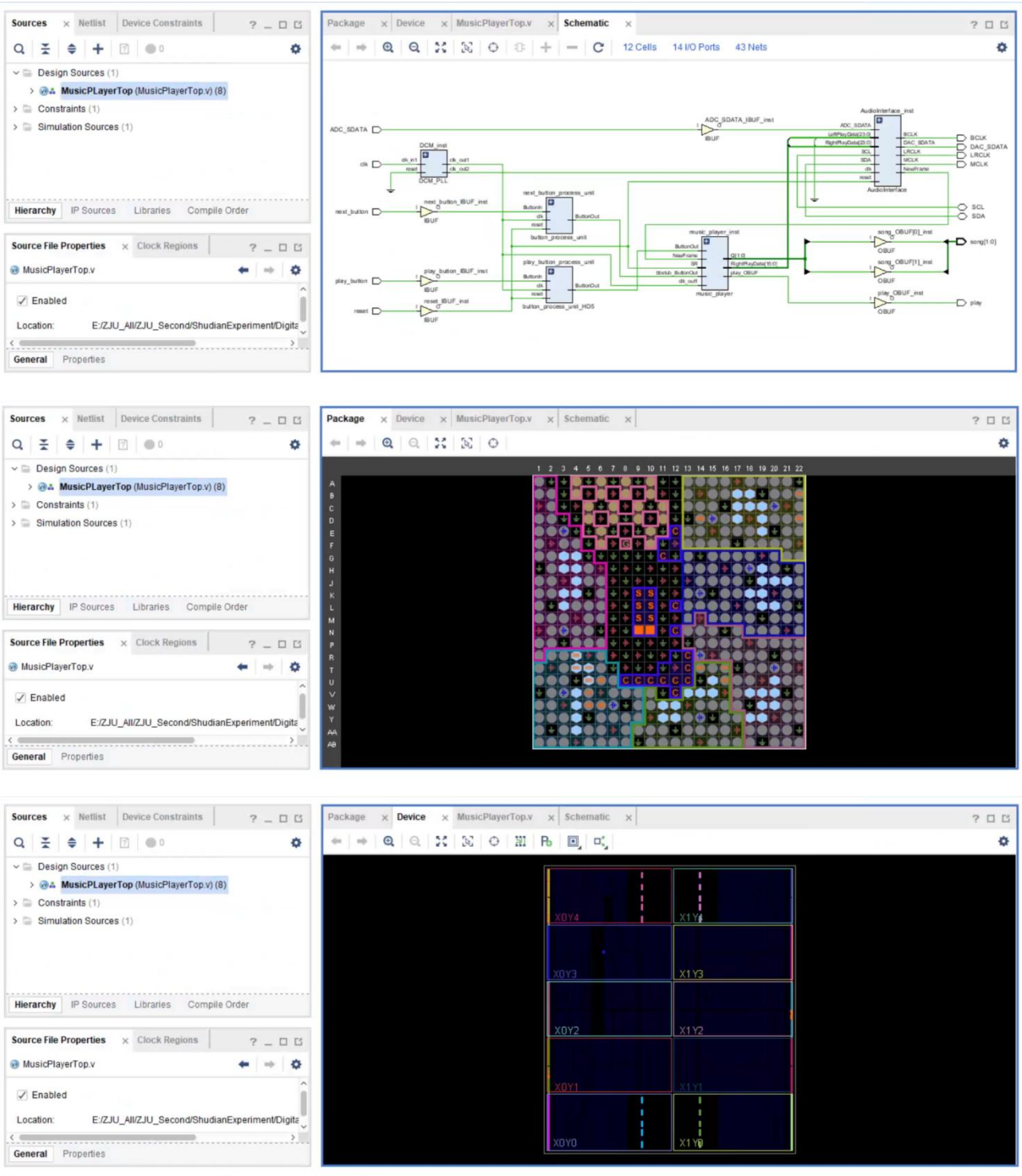
（1）IP 内核生成符合要求的 DCM 内核，需要配置 100MHz 的系统时钟和

12.50MHz 的音频时钟




```
wire sys_clk,audio_clk;
DCM_PLL DCM_inst
        (
         // Clock out ports
         .clk_out1(sys_clk),     // output clk_out1
         .clk_out2(audio_clk),     // output clk_out2
         // Status and control signals
         .reset(0), // input reset
         .locked(),       // output locked
        // Clock in ports
         .clk_in1(clk));      // input clk_in1
```

（2）RTL 分析与综合

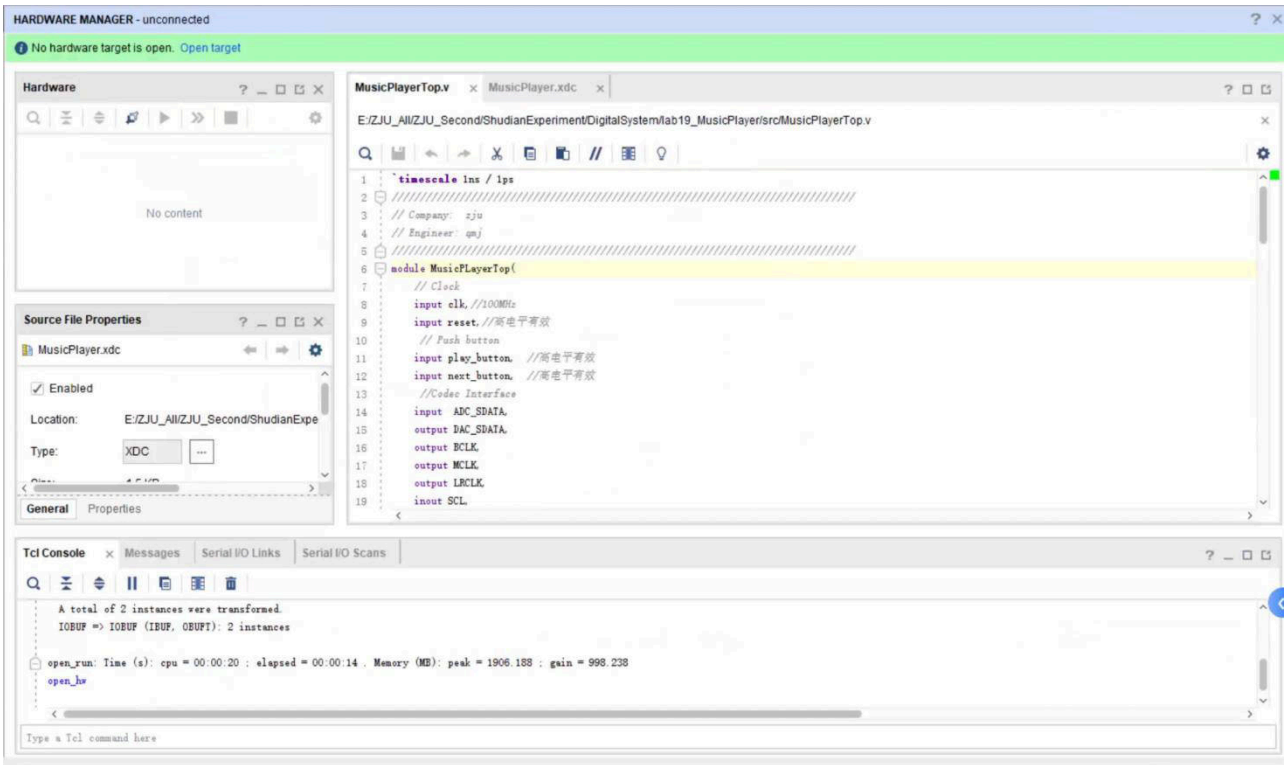


（3）引脚约束和时钟约束：按照课本要求进行约束（已经提供了.xdc 文件）

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type	Off-Chip Term
Scalar ports (12)												
ADC_SDATA	IN		T4	✓	34	LVCMOS33*	3.300				NONE	NONE
BCLK	OUT		T5	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
clk	IN		R4	✓	34	LVCMOS33*	3.300				NONE	NONE
DAC_SDATA	OUT		W6	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
LRCLK	OUT		U5	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
MCLK	OUT		U6	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
next_button	IN		D22	✓	16	LVCMOS33*	3.300				NONE	NONE
play	OUT		T14	✓	13	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
play_button	IN		D14	✓	16	LVCMOS33*	3.300				NONE	NONE
reset	IN		B22	✓	16	LVCMOS33*	3.300				NONE	NONE
SCL	INOUT		W5	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50
SDA	INOUT		V5	✓	34	LVCMOS33*	3.300		12	SLOW	NONE	FP_VTT_50

```
create_clock -period 10.000 -name clk -waveform {0.000 5.000} -add
set_property IOSTANDARD LVCMOS33 [get_ports BCLK]
set_property IOSTANDARD LVCMOS33 [get_ports DAC_SDATA]
set_property PACKAGE_PIN U5 [get_ports LRCLK]
set_property IOSTANDARD LVCMOS33 [get_ports LRCLK]
set_property PACKAGE_PIN U6 [get_ports MCLK]
set_property IOSTANDARD LVCMOS33 [get_ports MCLK]
set_property PACKAGE_PIN W5 [get_ports SCL]
set_property IOSTANDARD LVCMOS33 [get_ports SCL]
set_property PACKAGE_PIN V5 [get_ports SDA]
set_property IOSTANDARD LVCMOS33 [get_ports SDA]
set_property PACKAGE_PIN T4 [get_ports ADC_SDATA]
set_property IOSTANDARD LVCMOS33 [get_ports ADC_SDATA]
```

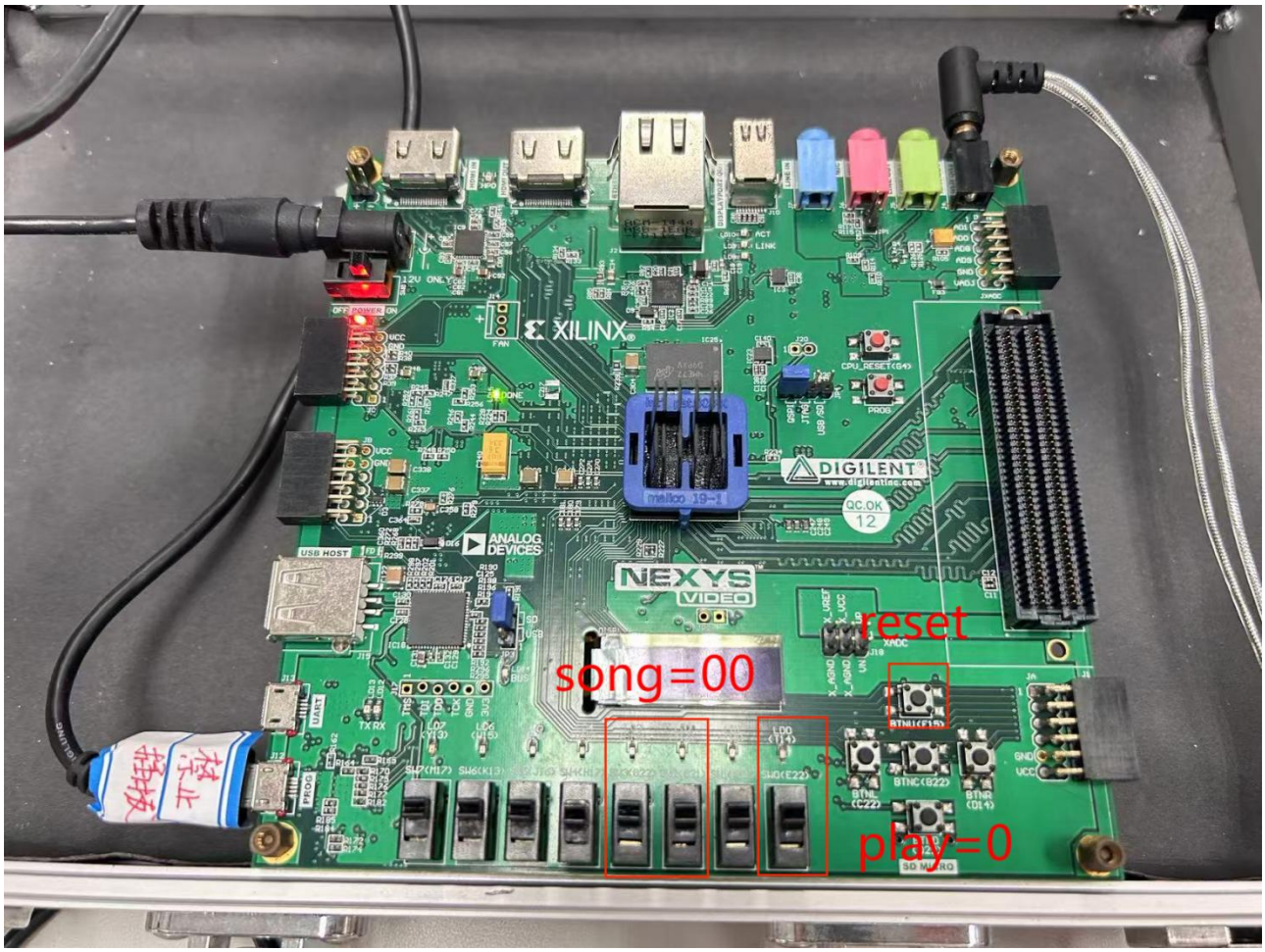
（4）连接开发板



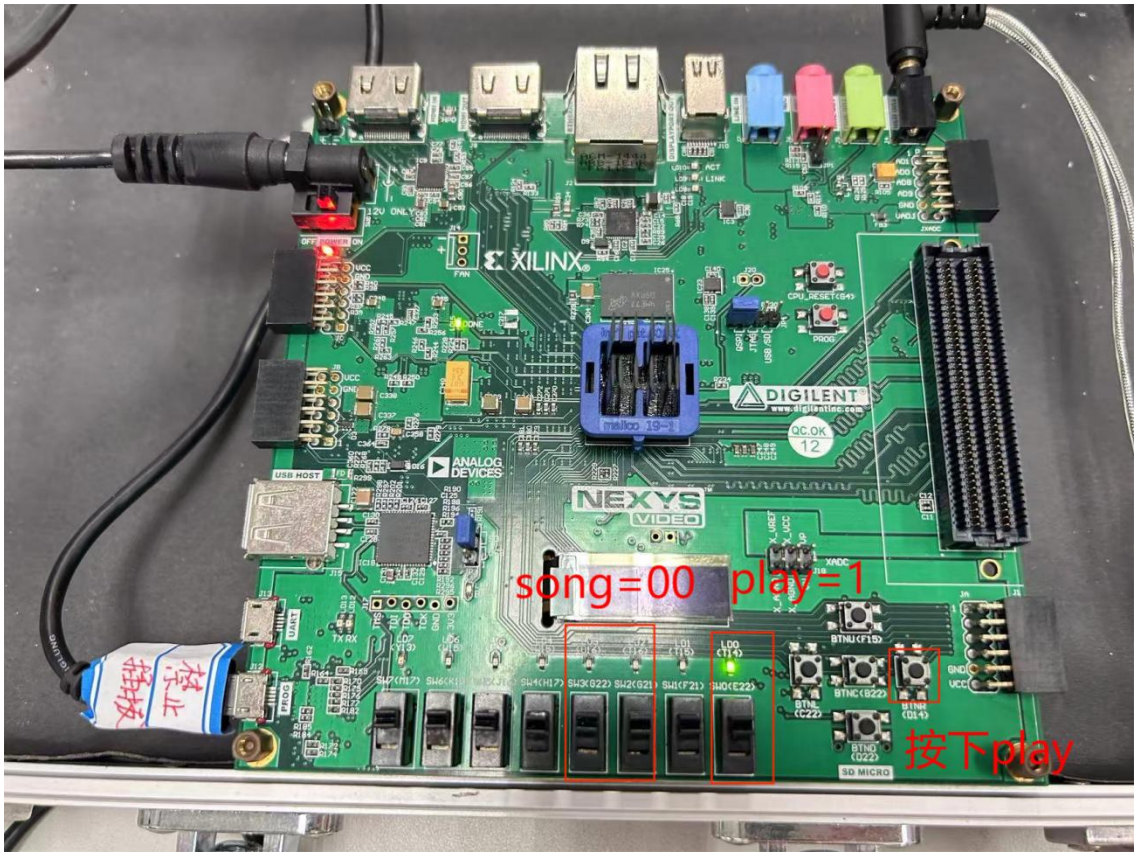
2．开发板测试：以下为 button_process_unit 防颤动功能测试

（1）引脚分析：reset 按键为中间按钮，next_button 按键为下面按钮，play_button 按键为右边按键；输出，play 为 LED0 指示灯，song 的序数由 LED2 和 LED3 指示，因此可以存储 4 首歌曲，其中 LED2 表示低位，LED3 表示高位

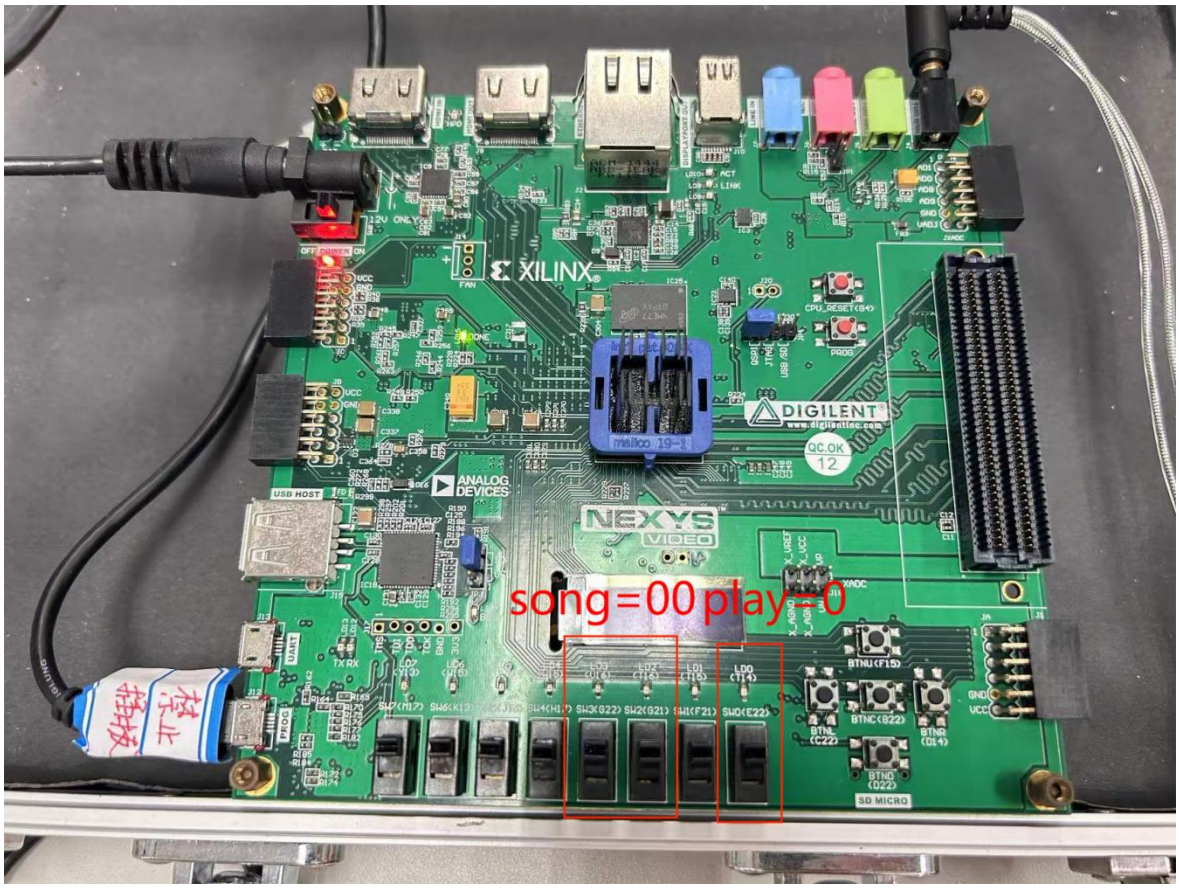
（2）执行 programming device 后，LED0 和 LED2、3 均熄灭，按下 reset 复位后，仍然处于熄灭状态



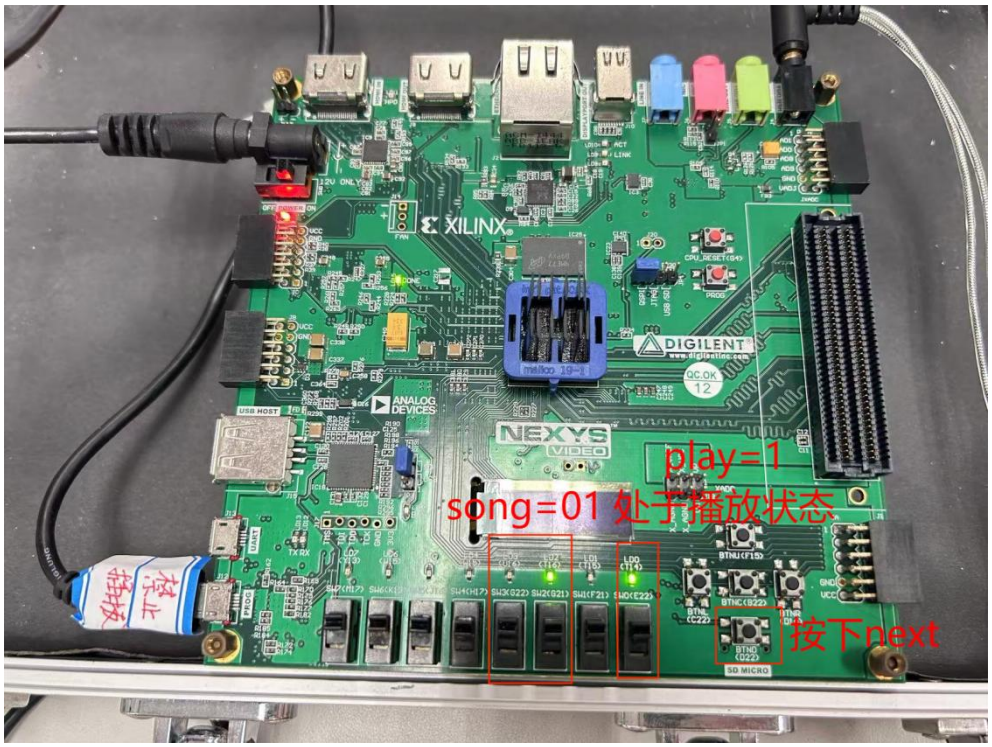
（3）第一次按下 play 按键，LED0 指示处于播放状态，LED2、3 熄灭，表明 song=00，播放第一首歌曲中



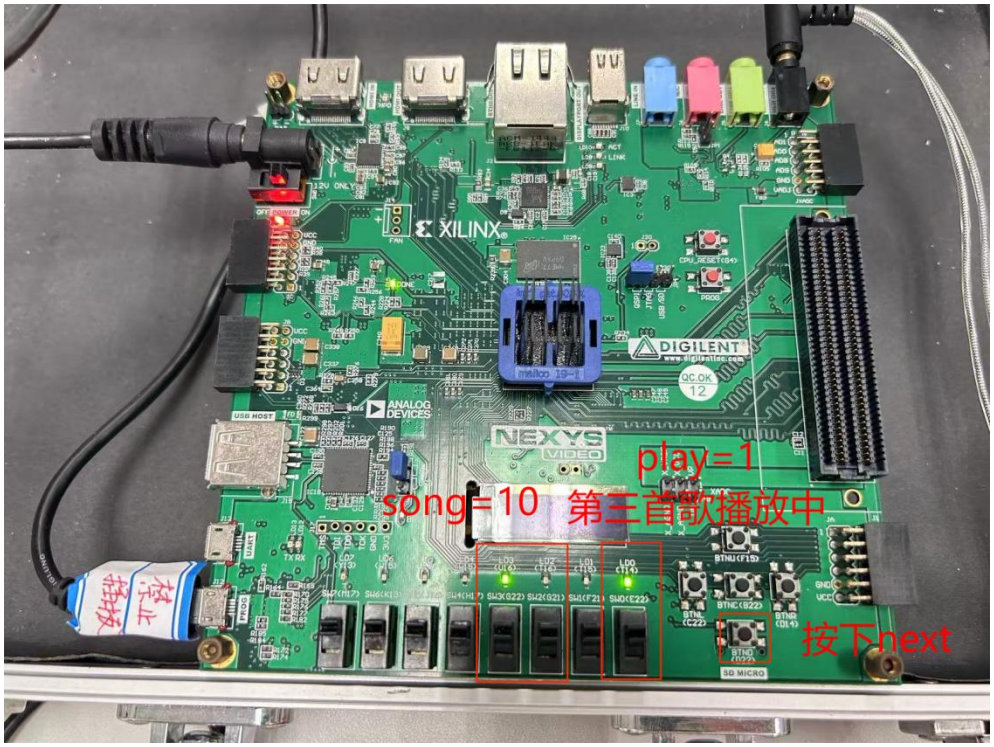
（4）当第一首歌播放完毕后，LED0、2、3 均熄灭，表明歌 00 已经播放完毕



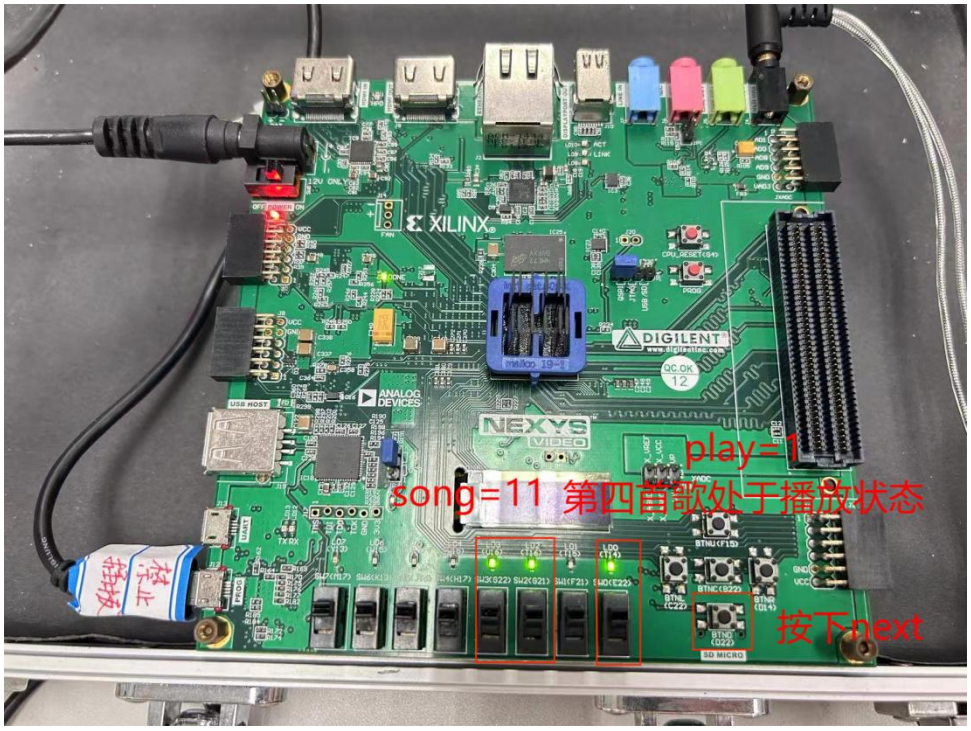
（5）按下 next 按键，LED2 亮，LED3 熄灭，LED0 亮，表明歌 song=01，play=1，第二首歌处于播放状态



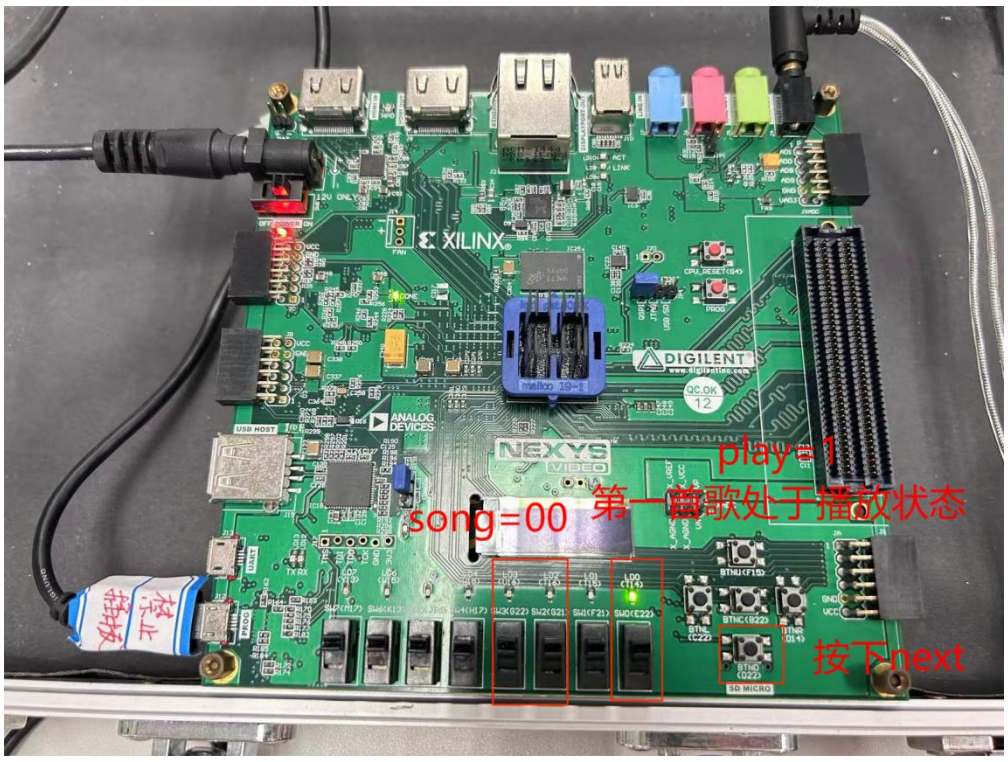
（6）再次按下 next，LED2 熄灭，LED3 亮，LED0 亮，表明歌 song=10，play=1，第三首歌处于播放状态



（7）再次按下 next，LED2、LED3 亮，LED0 亮，表明歌 song=11，play=1，第四首歌处于播放状态

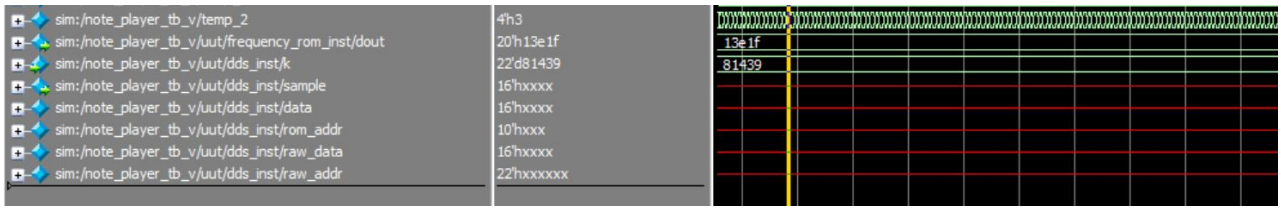


（8）再次按下 next，LED2、LED3 熄灭，LED0 亮，表明歌 song=00，play=1，重新回到第一首歌，处于播放状态



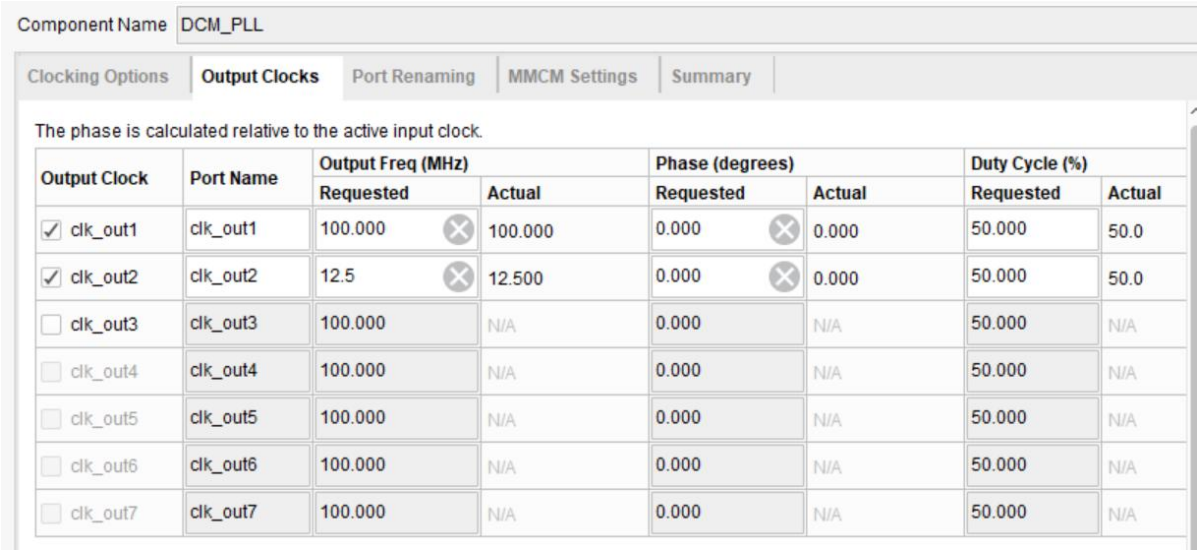
六、实验过程中遇到的问题和解决方案

1．遇到的问题首先是 note_player 中 sample 是不定态，而且 k 是确定的 note_player 中 dds 剩下的好多参数都是不定态，但是检查过后 lab15 的 dds 设计的是没有问题的，但是还是不定态，经过检查后发现，是在顶层代码中调用 dds 模块时，下意识认为 reset 时复位，只需要默认时 1'b0，但是在结构框图中，DDS 的 reset 端是由 play_enable 和 reset 共同决定的，当 play_enable=0，不播放时，dds 也处于复位状态




```
dds dds_inst(.k({2'b00,temp_dout}),.clk(clk),.reset(/* 1'b0 */~play_enable|reset),.samplin
```

2．第二个遇到的问题，主要是 vivado 工程建立过程中，要求生成符合要求的 DCM 内核，我在建立内核后，在配置 OutputClock 时，一开始我写的 PortName 是指导书中所要求的 sys_clk 和 audio_clk，仿真过程中发现了报错，但后面才发现代码中，PortName 这一栏是端口名，而不是实际连接的，后面才修改为 clk_out1，clk_out2



Output Clock	Port Name	Output Freq (MHz)		Phase (degrees)		Duty Cycle (%)	
		Requested	Actual	Requested	Actual	Requested	Actual
☑ clk_out1	clk_out1	100.000	100.000	0.000	0.000	50.000	50.0
☑ clk_out2	clk_out2	12.5	12.500	0.000	0.000	50.000	50.0
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A

```
DCM_PLL DCM_inst
        (
         // Clock out ports
         .clk_out1(sys_clk),     // output clk_out1
         .clk_out2(audio_clk),     // output clk_out2
         // Status and control signals
         .reset(0), // input reset
         .locked(),       // output locked
        // Clock in ports
         .clk_in1(clk));      // input clk_in1
//  ******************************************************************************
//      Button processor units
//  ******************************************************************************
```

七、思考题

1．在实验中，为什么 next_button、play_pause_button 两个按键需要消颤动及同步化处理，而 reset 按键不需要消颤动及同步化处理

答：（1）消除颤动：next_button 和 play_pause_button：这些按键用于功能操作（如切换歌曲、播放/暂停），机械触点抖动会导致短时间内多次电平变化。若不消抖动，系统可能误判多次操作；reset 按键：复位信号是电平敏感（而非边沿敏感），只需在有效电平（如低电平）保持足够时间即可完成复位。即使存在抖动，只要有效电平持续时间足够长（远大于抖动周期），复位功能仍能正确执行。

（2）同步化处理：next_button 和 play_pause_button 这类功能按键，异步输入可能产生亚稳态导致逻辑错误，同步化可以降低亚稳态风险；reset 复位信号，优先级一般最高，对时序要求宽松。

2．在主控制器 mcu 设计中，是否存在接收不到按键信息？若存在，概率多大？有没有必要修改设计？

答：mcu 设计中接收的按键信号已经通过外部按键处理模块，采用双触发器同

步和 10ms 的消抖动定时，亚稳态概率很低，确保信号稳定。可能存在的问题是未明确处理同时按下多个按键的矛盾，如同时按下 next 和 play_pause，可能导致状态跳转异常